

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

*I Jeevan Raaja S (192421005) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Jeevan Raaja S

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

COMPUTER ARCHITECTURE

Jeevan Raaja S

192421005

CSA1221

Dr. Vijaya Bhaskar K

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

ANSWER

- The CPU executes instructions and processes video data from cameras.
- Memory stores video frames and temporary processing data.
- I/O devices continuously transfer video streams to memory.
- Heavy data transfer creates bus congestion during peak hours.
- In a single-bus architecture, CPU, memory, and I/O share one communication path, causing delays.
- A multi-bus architecture allows simultaneous data transfers, improving throughput.
- Faster instruction execution reduces processing time for each video frame.
- Techniques like instruction pipelining, cache memory, and DMA improve system performance.

In a smart city surveillance system, cameras continuously send video data to the computer. The I/O devices transfer this data to memory, while the CPU processes it to detect threats. During peak hours, the large amount of video data creates congestion on the system bus, increasing delays. In a single-bus structure, all components share one bus, resulting in slower communication. A multi-bus structure provides separate communication paths, allowing multiple transfers simultaneously and reducing waiting time. Performance can be further improved by using pipelining, cache memory, Direct Memory Access (DMA).

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

ANSWER

- The processor follows the Fetch–Decode–Execute cycle.
- High CPI (Cycles Per Instruction) increases execution time.
- Frequent memory access causes CPU stalls.
- Branch instructions lead to pipeline delays.
- Cache memory reduces memory access time.
- Branch prediction minimizes branch penalties.
- Instruction pipelining allows multiple instructions to execute simultaneously.
- Superscalar processors and out-of-order execution further improve CPU performance.

A stock trading application processes millions of transactions every second. Every instruction passes through the Fetch, Decode, and Execute stages. Frequent memory accesses force the CPU to wait for data, while branch instructions interrupt the instruction pipeline, increasing the CPI. A high CPI results in slower transaction processing. This issue can be reduced by using cache memory to decrease memory latency, branch prediction to avoid pipeline stalls, instruction pipelining for parallel execution, and superscalar architecture to execute multiple instructions simultaneously. These improvements reduce execution time and increase CPU efficiency.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

ANSWER

- Addressing modes determine how data is accessed.
- Immediate addressing stores data directly in the instruction.
- Direct addressing accesses a specified memory location.
- Register addressing uses CPU registers.
- Register indirect addressing accesses memory through register pairs.
- Incorrect addressing modes may access wrong sensor values.
- Wrong memory addresses produce incorrect temperature readings.
- Proper use of 8085 instructions improves system reliability.

In an 8085-based temperature control system, sensors provide temperature data that the processor reads using different addressing modes. If an incorrect addressing mode is used, the processor may read data from the wrong memory location or register, resulting in incorrect temperature values. Such errors can cause improper operation of actuators. By selecting the correct addressing mode and using suitable instructions such as MOV, MVI, LDA, STA, ADD, and CMP, the processor can accurately read sensor data, compare values, and control industrial equipment reliably.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

ANSWER

- The 8086 divides memory into Code, Data, Stack, and Extra segments.
- CS stores program instructions.
- DS stores variables and data.
- SS stores stack information like function calls and return addresses.
- Incorrect segment register values cause invalid memory access.
- Stack overflow occurs when excessive data is pushed onto the stack.
- Proper segment initialization prevents memory errors.
- Correct addressing modes ensure accurate program execution.

The 8086 microprocessor uses segmented memory to efficiently manage its 1 MB address space. The Code Segment (CS) contains program instructions, the Data Segment (DS) stores variables, and the Stack Segment (SS) manages function calls and temporary data. If segment registers are loaded with incorrect values, the processor accesses the wrong memory locations, causing data corruption. Stack overflow occurs when the stack exceeds its allocated memory. Proper initialization of segment registers, careful stack management, and correct addressing modes ensure accurate instruction execution and reliable program performance.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

ANSWER

- Computers internally process binary data.
- Hexadecimal provides a compact representation of binary values.
- Incorrect conversion changes data and instructions.
- Conversion errors cause incorrect packet processing.
- CPU may execute wrong instructions due to incorrect binary values.
- Efficient CPU design detects processing errors quickly.
- Benchmarking measures CPU performance and identifies bottlenecks.
- Automated conversion tools reduce human errors.

In data communication systems, hexadecimal numbers are commonly used because they provide a shorter and easier representation of binary data. Before processing, the CPU converts hexadecimal values into binary. If a software engineer performs an incorrect conversion, the processor may interpret packet information incorrectly, leading to wrong instruction execution and communication errors. Efficient CPU architectures, together with benchmarking techniques, help identify performance issues and processing errors quickly. Using reliable conversion algorithms and validation mechanisms improves the accuracy and reliability of real-time communication systems.